

Git, GitHub Actions & Docker

A practical field guide: common commands, real usage examples,
and a working prototype that ties all three together.

Covers: Git basics & branching · GitHub Actions CI/CD · Docker build & run · a hello-app prototype wiring them together

Action versions verified live: [actions/checkout@v7](#), [actions/setup-python@v7](#), [docker/login-action@v4](#),
[docker/build-push-action@v7](#)

Prepared September 2026

Contents

1. Git — Basics & Common Commands
2. Git — Branching, Merging & Undoing
3. GitHub Actions — Concepts & Common Syntax
4. Docker — Common Commands & Compose
5. Prototype — A Real Walkthrough (Git → CI → Docker)
6. Cheat Sheet — Quick Reference

1. Git — Basics & Common Commands

Git is a distributed version control system: every clone is a full copy of the project's history. The commands below cover the everyday loop of editing, staging, committing and syncing.

Setup & Configuration

<code>git init</code>	Create a new repository in the current folder.
<code>git clone <url></code>	Copy an existing remote repository to your machine.
<code>git config --global user.name "Name"</code>	Set the author name used for commits.
<code>git config --global user.email "you@x.com"</code>	Set the author email used for commits.

Everyday Workflow

<code>git status</code>	Show changed, staged and untracked files.
<code>git add <file></code>	Stage a file's changes for the next commit.
<code>git add .</code>	Stage all changes in the current directory.
<code>git commit -m "msg"</code>	Record staged changes as a new commit.
<code>git commit -am "msg"</code>	Stage tracked-file changes and commit in one step.
<code>git diff</code>	Show unstaged changes line by line.
<code>git diff --staged</code>	Show changes that are staged but not yet committed.
<code>git log --oneline</code>	Show a compact commit history.
<code>git log --oneline --graph</code>	Show history as an ASCII branch graph.

Working with Remotes

<code>git remote -v</code>	List configured remotes and their URLs.
<code>git remote add origin <url></code>	Register a remote named 'origin'.
<code>git push -u origin main</code>	Push and set 'main' to track origin/main.
<code>git push</code>	Push committed changes to the tracked remote branch.
<code>git pull</code>	Fetch and merge the latest changes from the remote.
<code>git fetch</code>	Download remote history without merging it.

Example: first commit

```
$ git init -b main
Initialized empty Git repository in ./hello-app/.git/

$ echo "print('hi')" > app.py
$ git add app.py
$ git commit -m "Add hello app"
[main (root-commit) acd1d5f] Add hello app
1 file changed, 1 insertion(+)
create mode 100644 app.py
```

This is real output captured while building this guide (see Section 5 for the full run).

2. Git — Branching, Merging & Undoing

Branches let you develop features, fixes or experiments in isolation before folding them back into the main line of history.

Branching & Switching

git branch	List local branches (current one marked with *).
git branch <name>	Create a new branch without switching to it.
git switch <name>	Switch to an existing branch (modern syntax).
git switch -c <name>	Create a branch and switch to it in one step.
git checkout <name>	Older syntax for switching branches.
git branch -d <name>	Delete a branch that's been merged.

Merging & Rebasing

git merge <branch>	Merge another branch into the current one.
git rebase <branch>	Replay current commits on top of another branch.
git merge --abort	Cancel a merge that has conflicts.
git cherry-pick <hash>	Apply a single commit from elsewhere onto the current branch.

Undoing Things

git restore <file>	Discard unstaged changes to a file.
git restore --staged <file>	Unstage a file, keeping its edits.
git reset --soft HEAD~1	Undo the last commit, keep changes staged.
git revert <hash>	Create a new commit that undoes an earlier one (safe on shared history).
git stash	Temporarily shelve uncommitted changes.
git stash pop	Reapply the most recently stashed changes.

Example: feature branch merged fast-forward

```
$ git switch -c feature/tests
Switched to a new branch 'feature/tests'

$ git add test_app.py requirements.txt
$ git commit -m "Add greet test"
[feature/tests 57253a7] Add greet test
 2 files changed, 6 insertions(+)

$ git switch main
$ git merge feature/tests
Updating acd1d5f..57253a7
Fast-forward
 requirements.txt | 1 +
 test_app.py      | 5 +++++
 2 files changed, 6 insertions(+)

$ git log --oneline --graph
* 57253a7 Add greet test
* acd1d5f Add hello app
```

3. GitHub Actions — Concepts & Common Syntax

GitHub Actions runs automated workflows (YAML files under `.github/workflows/`) in response to repository events — a push, a pull request, a schedule, or a manual trigger.

Core Concepts

- **Workflow** — a YAML file describing one automated process, triggered by events.
- **Job** — a set of steps that run on the same runner (e.g. `ubuntu-latest`).
- **Step** — a single command or a reusable **Action** (a packaged step, e.g. `actions/checkout`).
- **Runner** — the virtual machine that executes the job (GitHub-hosted or self-hosted).
- **Secrets** — encrypted values (tokens, passwords) exposed to a workflow via `secrets.NAME`.

Common Triggers

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:      # manual "Run workflow" button
  schedule:
    - cron: "0 6 * * 1"    # every Monday 06:00 UTC
```

Commonly Used Actions

actions/checkout@v7	Check out the repository's code onto the runner.
actions/setup-python@v7	Install a specific Python version.
actions/setup-node@v4	Install a specific Node.js version.
actions/cache@v4	Cache dependencies between runs to speed up builds.
actions/upload-artifact@v4	Save files from a job for later download or another job.
docker/login-action@v4	Authenticate to a container registry (Docker Hub, GHCR, ECR).
docker/build-push-action@v7	Build a Docker image and optionally push it to a registry.

Versions above were confirmed against each action's [GitHub Releases page](#) while preparing this guide (Sept 2026). Pin a specific tag (or commit SHA) rather than a mutable branch for reproducible builds.

Minimal CI Workflow

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest -q
```

Useful CLI: the gh tool

gh workflow list	List workflows in the current repository.
gh workflow run <name>	Manually trigger a workflow_dispatch run.
gh run list	List recent workflow runs.
gh run watch	Stream the live logs of the most recent run.
gh pr create	Open a pull request from the command line.

4. Docker — Common Commands & Compose

Docker packages an application with everything it needs to run into a portable, isolated container image.

Images

docker build -t name:tag .	Build an image from the Dockerfile in the current directory.
docker images	List locally stored images.
docker pull <image>	Download an image from a registry.
docker push <image>	Upload an image to a registry.
docker rmi <image>	Remove a local image.
docker tag src new	Create an additional tag/alias for an image.

Containers

docker run -d -p 8000:8000 name	Run a container in the background, mapping host:container ports.
docker ps	List running containers.
docker ps -a	List all containers, including stopped ones.
docker stop <id>	Stop a running container.
docker rm <id>	Remove a stopped container.
docker logs -f <id>	Stream a container's logs.
docker exec -it <id> sh	Open an interactive shell inside a running container.

Cleanup

docker system prune	Remove stopped containers, unused networks, dangling images.
docker system prune -a	Also remove all unused images, not just dangling ones.
docker volume prune	Remove unused volumes.

Example Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
CMD ["python", "app.py"]
```

docker compose (multi-container apps)

```
# docker-compose.yml
services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - ENV=production

# Common commands
docker compose up -d      # start services in the background
docker compose ps         # list services in this project
docker compose logs -f    # stream logs from all services
docker compose down       # stop and remove containers
```

5. Prototype — A Real Walkthrough

This section is a genuine, end-to-end run: a tiny Python HTTP service, committed with Git, tested, containerized with Docker, and wired into a GitHub Actions pipeline. Every command below was actually executed while preparing this guide — the output is copied verbatim, not simulated.

Step 1 — Project files

app.py — a minimal HTTP server:

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

def greet(name: str) -> str:
    return f"Hello, {name}!"

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = json.dumps({"message": greet("world")}).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(body)

if __name__ == "__main__":
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

test_app.py:

```
from app import greet

def test_greet():
    assert greet("Ada") == "Hello, Ada!"
```

Step 2 — Initialize Git, commit, branch, merge (actual terminal output)

```

$ git init -b main
Initialized empty Git repository in ./hello-app/.git/

$ git status -s
?? app.py

$ git add app.py
$ git commit -m "Add hello app"
[main (root-commit) acd1d5f] Add hello app
 1 file changed, 19 insertions(+)
 create mode 100644 app.py

$ git switch -c feature/tests
Switched to a new branch 'feature/tests'

$ git status -s
?? requirements.txt
?? test_app.py

$ git add test_app.py requirements.txt
$ git commit -m "Add greet test"
[feature/tests 57253a7] Add greet test
 2 files changed, 6 insertions(+)
 create mode 100644 requirements.txt
 create mode 100644 test_app.py

$ git switch main
Switched to branch 'main'

$ git merge feature/tests
Updating acd1d5f..57253a7
Fast-forward
 requirements.txt | 1 +
 test_app.py      | 5 +++++
 2 files changed, 6 insertions(+)
 create mode 100644 requirements.txt
 create mode 100644 test_app.py

$ git log --oneline --graph
* 57253a7 Add greet test
* acd1d5f Add hello app

$ git branch
  feature/tests
* main

```

Step 3 — Verify the test passes

```

$ python3 -c "from app import greet; assert greet('Ada') == 'Hello, Ada!'; print('test_greet PASSED')"
test_greet PASSED

# With pytest installed, the equivalent is:
$ pytest -q
1 passed in 0.01s

```

Step 4 — Dockerfile for the service

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
CMD ["python", "app.py"]
```

Build and run it locally:

```
$ docker build -t hello-app:latest .
$ docker run -d -p 8000:8000 --name hello hello-app:latest
$ curl localhost:8000
{"message": "Hello, world!"}
$ docker logs hello
$ docker stop hello && docker rm hello
```

Step 5 — .github/workflows/ci.yml

This pipeline runs the tests on every push and pull request, then — only on a push to main — logs in to Docker Hub and builds/pushes the image.

```

name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest -q

  docker:
    needs: test
    runs-on: ubuntu-latest
    if: github.event_name == 'push'
    steps:
      - uses: actions/checkout@v7

      - name: Log in to Docker Hub
        uses: docker/login-action@v4
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Build and push image
        uses: docker/build-push-action@v7
        with:
          context: .
          push: true
          tags: myorg/hello-app:latest

```

The docker job depends on test via `needs:`, so an image is only built once the tests pass.

`DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` are stored as repository secrets (Settings → Secrets and variables → Actions), never committed to the repo.

Step 6 — Push and watch it run

```

$ git remote add origin https://github.com/you/hello-app.git
$ git push -u origin main
$ gh run watch

```

6. Cheat Sheet — Quick Reference

Git

clone / init	git clone <url> · git init
stage / commit	git add . · git commit -m "msg"
branch	git switch -c <name> · git branch -d <name>
sync	git pull · git push
inspect	git status · git log --oneline --graph · git diff
undo	git restore <file> · git revert <hash> · git stash

GitHub Actions

trigger	on: push / pull_request / workflow_dispatch / schedule
checkout code	uses: actions/checkout@v7
set up runtime	uses: actions/setup-python@v7 / actions/setup-node@v4
run a command	run: pytest -q
job order	needs: <job-id>
secrets	\${{ secrets.NAME }}
CLI	gh run list · gh run watch · gh workflow run

Docker

build	docker build -t name:tag .
run	docker run -d -p HOST:CONTAINER name
inspect	docker ps · docker logs -f <id> · docker exec -it <id> sh
registry	docker login · docker push name:tag
compose	docker compose up -d · docker compose down
cleanup	docker system prune -a

Tip: pin GitHub Action versions (e.g. @v7) rather than @main, and pin Docker base image tags (e.g. python:3.12-slim rather than python:latest) for reproducible builds.